
Titulo del TFG
Title in english



Trabajo de Fin de Grado
Curso 2025–2026

Autor
Eva Paula Mik

Director
Ismael Sagredo Olivenza

Colaborador
Colaborador no definido. Usa \colaboradorPortada

Grado en Desarrollo de Videojuegos
Facultad de Informática
Universidad Complutense de Madrid

Titulo del TFG
Title in english

Trabajo de Fin de Grado en Desarrollo de Videojuegos

Autor
Eva Paula Mik

Director
Ismael Sagredo Olivenza

Colaborador
Colaborador no definido. Usa \colaboradorPortada

Convocatoria: *Junio 2026*

Grado en Desarrollo de Videojuegos
Facultad de Informática
Universidad Complutense de Madrid

12 de mayo de 2026

Dedicatoria

*A toda la gente que nos ha hecho llegar hasta
aquí,
un besazo.
A todos los que han estado a nuestro lado.*

Agradecimientos

Os amo!

Mik

Os amo!

Pau

Os amo!

Eva

Resumen

Titulo del TFG

Nuestro estudio se basa en la comparativa de distintos modelos de IA para hacer frente a la simulación de físicas costosas en proyectos 3D en motores de videojuegos.

Para ello creamos una simulación en Unity3D con un objeto con componente Cloth y una esfera que actúe como colisionador con la tela. Tras generar los datasets, los exportamos a cuadernos de Jupyter dónde los entrenamos con diferentes modelos en Pytorch (con soporte GPU): una red neuronal sencilla, una RNN, un Encoder-Decoder.

Finalmente exportamos las redes entrenadas a Unity mediante el paquete de Sentis para simular el movimiento modificando los vértices de una tela sin componente Cloth. Realizamos comparativas entre los distintos modelos, tanto en cuanto a precisión del modelo como a resultados visuales.

Observamos que el modelo que mejores resultados obtuvo en cuanto a tal métrica fue X.

Palabras clave

Inteligencia Artificial, Simulación física, Unity, Videojuegos, Optimización, PyTorch, ML

Abstract

Title in english

We need to do this.

Keywords

We need to do this.

Índice

1. Introducción	1
1.1. Motivación	2
1.2. Objetivos	2
1.3. Plan de trabajo	3
1.4. Metodología	4
1.5. Estructura de la memoria (propuesta)	4
2. Estado de la Cuestión	5
2.1. Simulación física en videojuegos	5
2.1.1. Motores de videojuegos	6
2.1.2. Motores de física	6
2.2. Simulación de telas	7
2.2.1. Cloth	7
2.2.2. El problema de optimización	8
2.2.3. Soluciones	9
2.3. Inteligencia Artificial en videojuegos	9
2.3.1. Librerías	9
2.3.1.1. Pytorch	9
2.3.1.2. ONNX	10
2.3.1.3. Sentis	10
2.3.2. Modelos de optimización de telas	10
2.3.2.1. Subspace Neural Physics: Fast Data-Driven Interac- tive Simulation	10
2.3.2.2. Cloth and Skin Deformation with a Triangle Mesh Based Convolutional Neural Network	11
2.3.2.3. PBNS: Physically Based Neural Simulation for Un- supervised Garment Pose Space Deformation	12
2.3.2.4. SNUG	12
2.3.2.5. Neural Cloth Simulation	12
2.3.2.6. HOOD: Hierarchical Graphs for Generalized Mode- ling of Clothing Dynamics	13

3. Descripción del Trabajo	15
3.1. El dataset	15
3.1.1. La simulación	15
3.1.2. Los parámetros	16
3.1.3. Generación del dataset	17
3.2. EntrenamientoPython	17
3.2.1. Los modelos	17
3.3. SimulacionUnity	18
3.4. ModelosFuncionales	18
3.5. PruebaTesters	18
4. Conclusiones y Trabajo Futuro	19
4.1. Conclusiones	19
4.1.1. Conclusiones de la búsqueda de datasets	19
4.1.2. Conclusiones de la blablabla	19
4.1.3. Limitaciones	19
4.2. Trabajo Futuro	19
Contribuciones Personales	21
Bibliografía	23
A. Carteles para la generación del dataset	27
B. Formulario de recogida de citas	31
C. Resultados de los distintos modelos CNN	33
C.1. Intervalo 0.2s	33
C.2. Intervalo 0.4s	35
C.3. Intervalo 0.6s	36
C.4. Intervalo 0.8s	38
C.5. Intervalo 1.0s	39
Glosario	41
Licencia de uso del documento	43
Licencia de uso del código fuente	45

Índice de figuras

1.1. Diagrama de Gantt del proyecto	4
3.1. Malla de 25 vértices con parte superior fija.	16
A.1. Cartel colgado en la facultad de informática	27
A.2. Cartel colgado en redes sociales	28
A.3. Cartel colgado en pantallas de la facultad	29
B.1. Formulario de recogida de citas (primera parte)	31
B.2. Formulario de recogida de citas (segunda parte)	32
C.1. Esquema del modelo CNN con 0.2s de intervalo	33
C.2. Matriz de confusión del modelo CNN con 0.2s de intervalo	34
C.3. Gráfico de entrenamiento del modelo CNN con 0.2s de intervalo (mejor val_accuracy = 0.28777)	34
C.4. Esquema del modelo CNN con 0.4s de intervalo	35
C.5. Matriz de confusión del modelo CNN con 0.4s de intervalo	35
C.6. Gráfico de entrenamiento del modelo CNN con 0.4s de intervalo (mejor val_accuracy = 0.49473)	36
C.7. Esquema del modelo CNN con 0.6s de intervalo	36
C.8. Matriz de confusión del modelo CNN con 0.6s de intervalo	37
C.9. Gráfico de entrenamiento del modelo CNN con 0.6s de intervalo (mejor val_accuracy = 0.44952)	37
C.10. Esquema del modelo CNN con 0.8s de intervalo	38
C.11. Matriz de confusión del modelo CNN con 0.8s de intervalo	38
C.12. Gráfico de entrenamiento del modelo CNN con 0.8s de intervalo (mejor val_accuracy = 0.40583)	39
C.13. Esquema del modelo CNN con 1.0s de intervalo	39
C.14. Matriz de confusión del modelo CNN con 1.0s de intervalo	40
C.15. Gráfico de entrenamiento del modelo CNN con 1.0s de intervalo (mejor val_accuracy = 0.56002)	40

Índice de tablas

Introducción

En los últimos años, el aumento en el tamaño y la complejidad en los videojuegos ha supuesto la imprescindible optimización y automatización de diversos aspectos de su desarrollo, tanto en el ámbito del renderizado como en la lógica interna de los sistemas. Particularmente la evolución de entornos 3D, con el objetivo de obtener gráficos más detallados, ha implicado un incremento significativo en el coste computacional. Una forma de afrontar estas simulaciones para reducir su impacto computacional, es utilizar modelos de ML que permiten aproximar el comportamiento de estos elementos reduciendo el coste.

Empresas pioneras en hardware y software han desarrollado tecnologías para mejorar el rendimiento gráfico incorporando esta clase de algoritmos. Entre ellas destacan: NVIDIA con DLSS (Deep Learning Super Sampling), donde tenemos el Resident Evil Requiem que usa DLSS 4; y AMD con FSR (FidelityFX Super Resolution). Asimismo, varias compañías lo incorporan en el propio curso del juego simulando comportamientos y mecánicas como EA (FIFA), Embark Studios (Arc Raiders) o Ubisoft con sus múltiples líneas de investigación con la IA. En este contexto, la aplicación de modelos de aprendizaje automático en simulaciones físicas se presenta de forma prometedora.

En el campo de la simulación física, con el incremento de la capacidad de cálculo de los sistemas actuales, se busca una mayor fidelidad en la recreación de las interacciones entre los diferentes elementos. Así por ejemplo, elementos como el cabello, los fluidos o las telas, elementos que hace poco más de una década estaban muy simplificados, en la actualidad se busca cada vez más con comportamiento realista en sus interacciones. Esta simulación física de sus comportamientos resulta en una carga computacional significativa, lo que afecta negativamente al rendimiento de los videojuegos, en especial en términos de tasa de fotogramas. Estamos en el momento peak de investigación y aplicación en industria de ML para solucionarlo.

1.1. Motivación

En la misma línea de otras tecnologías ya utilizadas en el desarrollo de videojuegos como el mencionado Ray Reconstruction de NVIDIA, donde se aplican modelos de redes neuronales profundas para reducir la carga computacional de los juegos en el renderizado, se conducen estudios y empiezan a aplicar cosas similares en la industria en simulación física, otro de los elementos más demandantes de cómputo en un videojuego. Nace en un contexto dónde los juegos tienden a ser distribuidos ya no solo en consolas con hardware y gráficas dedicadas, sino en múltiples plataformas con el objeto de maximizar ingresos, donde no todas tienen las mismas capacidades de cómputo. El uso de estas técnicas de optimización puede permitir recrear estas simulaciones con un grado de fidelidad aceptable, en dispositivos de menor potencia como puede ser Nintendo Switch, Handle pcs o dispositivos móviles.

Este enfoque implica una carga elevada al inicio de entrenamiento de modelos, compensado con una ejecución eficiente en tiempo real. Cabe destacar que, si bien utilizar la IA supone sacrificar la precisión absoluta de las físicas, no queremos caer en la falacia de la inmersión [Salen y Zimmerman \(2004\)](#). Nuestro objetivo es lograr simulaciones visualmente verosímiles que mantengan una percepción fiel de realismo, adoptando así una perspectiva cercana al diseño de videojuegos. Por ello, se ha escogido Cloth para la simulación de telas como caso de estudio debido a que representa un equilibrio adecuado entre accesibilidad tecnológica, complejidad física e impacto visual. De esta manera, realizaremos un análisis tanto técnico como perceptivo.

1.2. Objetivos

El objetivo general de este proyecto es la implementación de un modelo de IA que permita replicar el movimiento en tiempo real de una tela en un motor de videojuegos con baja latencia. Para cumplir con este objetivo, se plantean las siguientes metas durante el desarrollo del proyecto:

1. Generar una animación de una tela y un colisionador que varíe su movimiento en cada ejecución de la escena.
2. Generar y procesar datasets con la información relevante de la tela respecto a su entorno y colisionador.
3. Implementar, entrenar y evaluar los distintos modelos de aprendizaje automático, tales como redes neuronales (LSTM, CNN y RNN, GNN).
4. Analizar y comparar los resultados visuales de los distintos modelos.

1.3. Plan de trabajo

El desarrollo del proyecto se llevará a cabo en el periodo lectivo del curso 2025-2026. Dados los objetivos del apartado anterior, la planificación se divide en las siguientes iteraciones:

- **Iteración 0:** En esta etapa se concretan los objetivos reales de la investigación y se establece la planificación del trabajo respecto a la propuesta inicial.
- **Iteración 1:** Etapa de preparación, donde se cumplirán los siguientes subobjetivos para implementar un sistema de simulación adecuado para ser procesado por los modelos en un futuro.
 - **Iteración 1.1:** Se realizará una investigación de trabajos relacionados con nuestro tópico, relevante para dirigir el desarrollo de nuestros experimentos. Se analizarán distintas alternativas y se decidirá cuales resultarán más interesantes de replicar, se definirán qué tipo de datos deberemos recopilar para el entrenamiento y qué tipo de redes podrán llevarlo a cabo.
 - **Iteración 1.2:** Se creará una simulación básica en Unity y un sistema de registro de datos relevantes a la tela.
- **Iteración 2:** Se establecerá un entorno de trabajo para el entrenamiento de los datos recopilados. Se generará un pipeline claro en el que se integre la simulación de recogida de datos, su limpieza y posterior uso para entrenar el modelo y, finalmente, la incorporación de este al entorno de simulación. Para poder asegurarnos del correcto funcionamiento de este sistema se generará un modelo simple y se comprobará que el pipeline es correcto. En esta fase se comenzarán a redactar los primeros capítulos de la memoria.
- **Iteración 3:** Con una base sólida ya montada, se aumentará la complejidad y, por consiguiente, el coste de la simulación. Se implementarán distintos modelos y se llevarán a cabo pruebas con todos ellos para asegurarse de su fiabilidad.
- **Iteración 4:** Si las anteriores iteraciones funcionan de manera correcta esta última no tendrá ninguna carga de implementación. Se alcanzará la última meta definida: hacer distintas pruebas entrenando los modelos ya desarrollados, sacar métricas de todos ellos para poder hacer comparaciones y hacer un análisis exhaustivo de estos resultados, sacando así las conclusiones de la investigación.

El desarrollo de las tareas y su planificación se puede ver en la figura [1.1](#).

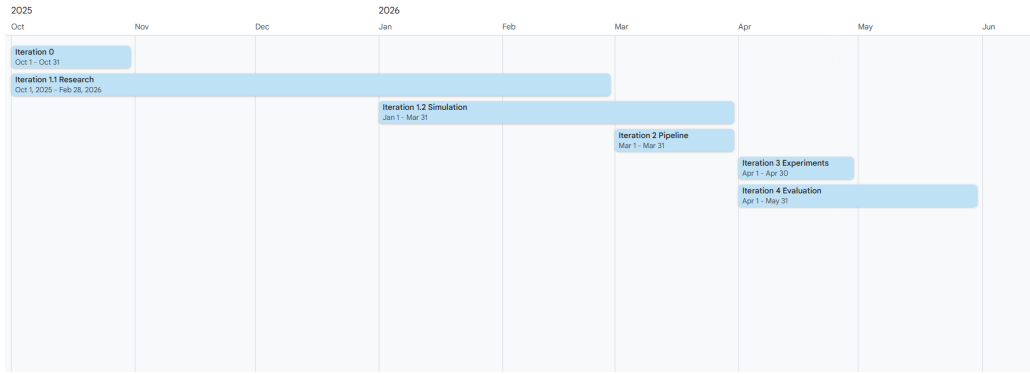


Figura 1.1: Diagrama de Gantt del proyecto

1.4. Metodología

Para el desarrollo de esta investigación se emplearán herramientas de libre acceso y uso gratuito, con el objetivo de facilitar la reproducibilidad de los resultados obtenidos. El entorno de desarrollo principal será Windows 11, al tratarse del sistema operativo utilizado por el equipo de trabajo y extendido entre los potenciales usuarios. La simulación de la tela, así como la integración del modelo de inteligencia artificial, se llevarán a cabo en Unity (versión 6000.3.2f1, correspondiente a una versión de soporte a largo plazo). En cuanto a los lenguajes de programación, se empleará C# para el desarrollo dentro de Unity, incluyendo la generación de la simulación y la recopilación de datos, mientras que se utilizará Python con Pytorch para el entrenamiento de los modelos de aprendizaje automático. La integración de los modelos entrenados en el entorno de ejecución se realizará a través del paquete Unity Sentis.

Para la gestión del código fuente y el trabajo colaborativo se utilizará GitHub, basado en el sistema de control de versiones Git. Asimismo, se empleará la herramienta GitHub Projects para la planificación, asignación y seguimiento de tareas. Como herramientas adicionales, se utilizará Google Drive para el almacenamiento de documentación, apuntes de reuniones y materiales de trabajo puntuales, y Discord para la coordinación del equipo mediante reuniones online, además de encuentros presenciales semanales.

1.5. Estructura de la memoria (propuesta)

Capítulo 2

Estado de la Cuestión

En este capítulo se presenta una breve resumen del desarrollo de las simulaciones físicas en videojuegos, así como el hardware y software que ha hecho que esta evolución sea posible. También se presenta la simulación de telas, sus diferencias de implementación, los problemas de optimización que estas suponen y los avances en modelos de IA que permiten imaginar otras soluciones.

2.1. Simulación física en videojuegos

Basta con remontarse a la creación del Pong en 1972, para darse cuenta de la importancia que las simulaciones físicas han tenido en el ámbito de los videojuegos desde el primer momento. Las físicas permiten crear situaciones del gameplay que no están predefinidas, enriqueciendo inmensamente la experiencia del jugador.

No obstante, en el inicio del desarrollo de videojuegos las técnicas para representar comportamientos físicos eran todavía rudimentarias: se diseñaban las interacciones físicas previamente para parecer realistas, ya que dada la capacidad de cómputo con la que solían contar las máquinas, era imposible hacer todos los cálculos físicos en tiempo real. Se realizaban detecciones de colisiones simple, sin uso de fuerzas y descripciones de movimientos o respuestas de cada elemento del juego en lugar de sistemas de física.

Estos métodos eran claramente limitados en la precisión que ofrecían, y quedaba clara la necesidad de una solución más general y reproducible. El mismo Ian Millington nos introduce en su libro *Game Physics Engine Development* [Millington \(2007\)](#) el juego Exile, creado por Peter Irvin y Jeremy Smith; si bien este juego fue publicado en 1988, ya contaba con conservación del momento en colisiones, gravedad, y fuerzas como el viento o explosiones. Lo resaltamos como uno de los primeros juegos que cuenta con un motor de físicas completo.

Actualmente encontramos juegos con simulaciones físicas muy realistas como *x* o *y* donde más allá de satisfacer necesidades de gamplay, se consiguen mundos creíbles

en su conjunto.

2.1.1. Motores de videojuegos

Los motores físicos, sumados a otros elementos análogos como los motores de renderizado y sonido, crean el software al que nos referimos como motores de videojuegos. En el mercado encontramos establecidos motores como Unity o Unreal Engine, motores propios de grandes estudios y aquellos pequeños indies como Godot. Los motores con más presencia en el desarrollo indie son:

- **Unreal:** Es un motor de videojuegos creado por Epic Games en 1998. Destaca por su motor gráfico, que permite gráficos realistas en tiempo real con tecnologías como Nanite para geometría virtualizada o Lumen para iluminación dinámica. Es multiplataforma y lo encontramos no sólo en desarrolladores indie sino en grandes producciones AAA. Se caracteriza por su uso de Blueprints en programación con c++. Integraba Physx como motor físico hasta la introducción reciente de Chaos Physics en su versión 5.7, que aporta novedades que destacaremos en el apartado 2.2.3.
- **Unity:** Creado por Unity Technologies en 2005, abarca un porcentaje del 51 % de juegos publicados en 2024 y un 26 % [Tower \(2025\)](#) en las unidades vendidas en Steam. Es el motor más utilizado por desarrolladores indies y cuenta con documentación, gran comunidad y extensiones como Sentis para la integración de modelos de IA, que explicaremos más adelante. Unity3D integra PhysX como motor de físicas, pero puede incorporar otros sistemas de físicas como Havok bajo licencia [Technologies \(2026a\)](#). Está basado en la arquitectura Entidad-Componente en c#, y tiene un enfoque menos potente en cuanto a gráficos y tamaño del juego.

2.1.2. Motores de física

La creación de motores de física completos aportó a los videojuegos escalabilidad en su gameplay, manteniendo la coherencia de sus elementos físicos. Esto último resulta imprescindible para la inmersión del jugador [Hecker \(2000\)](#).

En los años 90, con la popularidad del 3D y gráficos cada vez más avanzados, la física era el próximo área en la que marcar la diferencia: la evolución de estos motores era inevitable. Los grandes cálculos en tiempo real con su correspondiente carga para la CPU plantearon dos grandes retos: el avance de los motores físicos cada vez más precisos y potentes, y, por otro lado, la optimización del hardware para llegar a la demanda computacional que esto suponía.

En 2002, se funda Ageia Technologies, responsable de la creación del primer procesador de física dedicado para ordenadores, denominado PhysX PPU (Physics Processing Unit), así como del motor PhysX [NVIDIA \(2026\)](#), que sigue siendo líder en la industria a día de hoy. Posteriormente, NVIDIA Corporation, conocida por su

papel en la producción de GPUs, adquiere Aegia, impulsando el cálculo de físicas en GPU. Esto, sumado a la dificultad de penetrar en el mercado del hardware, hace que la producción de PPU's quede obsoleta.

Por otro lado, Intel adquiere en 2007 Havok [Havok \(2026\)](#), otro de los primeros motores de físicas, cancelando el programa HavokFX que pretendía llevar el cálculo de físicas a la GPU para realizar estos en sus CPUs multinúcleo. Posteriormente es adquirido por Microsoft en 2015 y se integra en las herramientas de desarrollo como DirectX. Se establece como el middleware más común en grandes producciones y simulaciones complejas en estudios, mientras que Physx se convierte en libre acceso en 2018 y se integra en motores de videojuegos como Unreal.

Los motores físicos nacen en un contexto de competición entre empresas, quienes intentan establecer su hardware como óptimo para la simulación de físicas. La disputa y diversificación para carga computacional entre CPU y GPU la veremos durante todo el desarrollo de este proyecto.

2.2. Simulación de telas

Entendemos los objetos deformables o soft-bodies como aquellos que no son rígidos. Así como en los objetos rígidos podemos simular su movimiento sabiendo su trayectoria y centro de masa, para los objetos deformables necesitamos tener en cuenta todas las partículas y propiedades físicas que lo componen. Entre estos encontramos elementos como las telas, el agua o el viento [Kenny Erleben \(2005\)](#).

El protagonista de la simulación de telas es la vestimenta: vestidos, cintas o capas moviéndose dependiendo de las acciones del jugador, viento y condiciones específicas permiten que el jugador se inmerse por completo. Además de tener objetos hechos a partir de tela como banderas, podemos incluso aplicar las físicas de la tela en otros elementos como pelo, hierba o globos; como señala Havok: “*Havok Cloth is used to create secondary motion and to enhance any object that flows*”.

En esta sección veremos cómo se representa y simula una tela en tiempo real, el problema que supone y qué tipos de soluciones encontramos. Nos centraremos mayoritariamente en ejemplos de Unity3D, como nuestro referencia de motor de videojuegos.

2.2.1. Cloth

Cloth es un componente que nos proporciona Unity3D que funciona junto al componente Skinned Mesh Renderer para simular telas. Está basado en Cloth de PhysX y fue incorporado con el objeto de poder disponer de ropa para personajes. Tiene múltiples propiedades como la resistencia a la flexión, rigidez, damping o uso de gravedad.

Se trata la tela como una malla de partículas conectadas por restricciones. El Skinned Renderer calcula las transformaciones basándose en una malla de baja resolución sobre la que realiza los movimientos en los vértices, que dinámicamente se aplican en la malla real y obtenemos la simulación visual. Aquellos constraints no visibles o de distancia 0, fijos, no serán calculados. La tela cuenta con capacidad de colisionar internamente y con dos tipos de colliders externos: cápsulas o esferas. Cualquier objeto externo con el que se quiera colisionar tendrá que ser un conjunto de estos colliders.

Pese a encontrar múltiples acepciones en el término solver para distintas resoluciones de problemas, aquí nos referiremos como solver al algoritmo computacional que se encarga de predecir el comportamiento del objeto deformable en renderizados y simulaciones físicas, específicamente, de telas. Encontramos infinitos tipos de solvers. Los primeros solvers se basan en algoritmos que emplean la fuerza bruta: realizan cálculos en masa para sistemas de muelles (MSS o Mass Spring Systems) basándose en la Ley de Hooke. Los vértices de la tela tienen un peso y están unidos por tres tipos de muelle que aseguran que la tela mantenga su forma, no se distorsione o doble en exceso [Provot \(1995\)](#). Los solvers más actuales resultan en el PBD, proceso iterativo que proyecta las posiciones de las partículas sobre un “espacio de restricciones” [Müller et al. \(2007\)](#), es decir, actualizando el desplazamiento generado por cada fuerza en cada vertice de la simulación [Chaudhary et al. \(2025\)](#).

El solver de PhysX se basa en PBD, es decir, trabaja directamente sobre las posiciones. Encontramos también XPBD [Macklin et al. \(2016\)](#), una versión extendida que puede incorporarse en Unity.

2.2.2. El problema de optimización

Como hemos introducido antes, la simulación de telas en tiempo real es un problema que persiste, en el que seguimos buscando la optimización a día de hoy. El aumento de tamaño o complejidad de una tela, con su alta carga computacional, conlleva el decaimiento inmediato de frames. A parte de dificultar el realismo en juegos y demás simulaciones a tiempo real, resulta un gran impedimento en entornos donde se requieren fotogramas por segundo altos como son los juegos en realidad aumentada.

Mientras que el Cloth de PhysX puede delegar la simulación a GPUs compatibles con CUDA, el Cloth integrado por defecto en Unity realiza todo su trabajo en la CPU, lo cual ralentiza mucho la ejecución. En el caso de Unreal, previamente al lanzamiento de ML Cloth Simulation, Chaos Physics delegaba su mayor fuerza de trabajo en la CPU.

2.2.3. Soluciones

Encontramos varios tipos de soluciones para la optimización de la simulación de telas:

- Optimización por CPU: encontramos la extensión de Obi Cloth [Method \(2019\)](#), que aprovecha el sistema de Jobs de Unity para paralelizar la simulación y acelerar la simulación sin salir de la CPU. Es uno de los paquetes más populares para solucionar este problema.
- Paralelización en GPU: a través de shaders se delega la fuerza de cálculo del skinning en la GPU. Encontramos diferentes estudios que aportan diferentes enfoques para la optimización: Jim Hugunin realiza una investigación para alcanzar altas resoluciones y evitar colisiones en las mallas [Jim Hugunin \(2016\)](#), mientras que otros se centran en la mejora de rendimiento pura para adaptar simulaciones a entornos VR [Va et al. \(2021\)](#).
- Entrenamiento con IA: el enfoque más actual de la resolución de simulaciones complejas como Cloth es la Inteligencia Artificial. Se entrenan modelos de ML y DL con los datos de las mallas animadas o poses, con lo que obtenemos más adelante mejor rendimiento en tiempo real. Se sacrifica memoria y tiempo de entrenamiento para obtener mejores y mucho más rápidas simulaciones de tela en escena. Actualmente, nos encontramos en el paso de investigación a implementación en grandes empresas: Unreal ya proporciona una simulación de tela a través de aprendizaje automático [Games \(2025\)](#), en el que destaca su realismo. Aún así, cuenta con limitaciones al basarse en NearestNeighborSearch: “el sistema es capaz de predecir detalles como arrugas pero no movimiento dinámico como ondulaciones o drapeados”. En la próxima sección detallaremos las numerosas investigaciones recientes que abarcan aspectos desde la reproducción de arrugas, el compromiso entre calidad, coste y rendimiento hasta colisiones y penetraciones en las mallas.

2.3. Inteligencia Artificial en videojuegos

Como se ha introducido previamente, en la industria del videojuego, como en muchas otras, la atención está fijada en la inteligencia artificial. En este apartado presentaremos los recursos e investigaciones más relevantes para el desarrollo de este proyecto.

2.3.1. Librerías

Estas son las librerías que han resultado indispensables para entrenar los modelos de IA y conectarlos a la simulación en Unity.

2.3.1.1. Pytorch

Pytorch es una de las librerías de código abierto que permite escribir redes neuronales propias. Los tensores, matrices multi-dimensionales análogas a los NumPy

arrays de Python, permiten optimizar las redes tanto para el uso de CPU como de GPU [Contributors \(2026\)](#). Suele ser la más elegida en entornos académicos, ya que permite prototipado rápido y entrenamiento dinámico. Fue originalmente desarrollado por Facebook, y todavía cuenta con una enorme comunidad de desarrolladores.

2.3.1.2. ONNX

ONNX es un formato creado para representar modelos de aprendizaje automático [Project \(2026\)](#). Fue introducido por Facebook y Microsoft en 2017, con el objetivo de facilitar el cambio de entornos de trabajo en aprendizaje automático.

2.3.1.3. Sentis

Sentis (previamente Inference Engine) es una librería de inferencia de redes neuronales para Unity, predecesora introducida en el 2023 de la librería Barracuda. Permite importar modelos de redes neuronales entrenados a Unity y ejecutarlos en tiempo real tanto con la CPU como con la GPU [Technologies \(2026b\)](#).

2.3.2. Modelos de optimización de telas

Desde la introducción del PBD, la optimización de simulaciones con redes neuronales cuenta ya con un amplio recorrido. En este apartado se describen los estudios más destacados de los últimos años. Son también inspiración directa para la implementación de nuestros experimentos, con el objetivo de descubrir cuáles de estas técnicas son verdaderamente aplicables a menor escala, en desarrollo indie con recursos limitados en cuanto a hardware y especialización físico-matemática.

2.3.2.1. Subspace Neural Physics: Fast Data-Driven Interactive Simulation

Este desarrollo de la unidad de investigación Ubisoft La Forge [Holden et al. \(2019\)](#) implementa la primera técnica que incorpora reducción de modelo o el uso de subespacios con colisiones con objetos externos en la tela. El estudio se centra en conseguir un balance entre el rendimiento en tiempo real y el coste y memoria necesarios para realizar el precómputo.

El entrenamiento se realiza sobre las posiciones de una simulación realizada en Maya aplanados en un vector, al que se añade otro vector plano con los parámetros necesarios como posición del suelo, valor del viento, o collisionadores con la tela. Se aplica un PCA sobre ambos denotando cuántos ejes de deformación de la tela se tienen en cuenta para representar la complejidad de la recreación: 64, 128, 256; siendo este último el que conserva casi el mismo nivel de detalle que la simulación original.

Para evitar que la simulación colapse con el error acumulado aplican dos estrategias: ventanas de frames superpuestas, con bloques de 32 frames con mini-batches

para mantener la coherencia a largo plazo, y ruido gaussiano para que la red aprenda a corregirse. Además, los cálculos necesarios durante la ejecución se aceleran con shaders en GPU (GPU decompression) y se recalculan las normales de los vértices necesarias para el renderizado.

Sus principales limitaciones residen en la necesidad de quitar a mano errores producidos en la simulación, falta de pruebas en objetos plásticos, posibles colisiones internas fallidas, y limitación de objeto colisionador fijo en el espacio y sin contar con objetos cayendo por el espacio. Se necesitaron 10^6 frames de simulación para entrenamiento para mejores resultados, así como períodos de entre 10 y 48 horas de entrenamiento.

2.3.2.2. Cloth and Skin Deformation with a Triangle Mesh Based Convolutional Neural Network

Este estudio [Chentanez et al. \(2020\)](#) es un ejemplo de uso de CNN para la simulación de cloth. Reseña la dificultad de integrar una CNN debido a la construcción de la malla, incompatible antes de su modificación con una cuadrícula 2D. Resuelve el problema entrenando mallas "manifold" triangulares: es decir, mallas cuyos bordes están conectados por al menos dos caras (menos los extremos) dejando una separación entre exterior e interior. Mapear la malla a una imagen 2D es lo que permite realizar la convolución.

Crean una red encoder-decoder formada por bloques "UpConv" and "Down-Conv" con operadores nuevos de convolución, pooling y unpooling. Destacamos como reduce la dimensionalidad de la malla en el pooling: colapasa los edges siguiendo un método en espiral. También señala la importancia de usar Leaky-CNN: para no dejar neuronas muertas, que resulta un problema en la simulación. Además de usar L2 con la L1 para que no se quede el error acumulado en ciertos vértices.

Este estudio, a diferencia del anterior, incluye brazos y manos, y se rige en un marco de posiciones locales por cada triángulo. Afirma conseguir resultados muy cercanos al *ground truth* usando la base de animaciones *Berkley animation base* y contando con PC potente. No obstante, resaltan una vez más los largos tiempos de entrenamiento (entre 4 y 20 horas) y la necesidad de calidad y cantidad de datos de entrenamiento. Tampoco aporta simulaciones para topología dinámica y se ciñe mucho al entrenamiento.

Otro ejemplo de estudio que usa una CNN para la simulación de telas es DeepSD: Real-time Cloth Simulation with Deep Learning [Bertiche et al. \(2021b\)](#); en la que proyecta la malla 3D en un plano utilizando sus coordenadas UV.

2.3.2.3. PBNS: Physically Based Neural Simulation for Unsupervised Garment Pose Space Deformation

PNBS Bertiche et al. (2021a) presenta una simulación neuronal DL no supervisada, como alternativa al enfoque clásico PBS. Integra principios físicos en el entrenamientos con funciones físicas de energía, entre ellas restricciones de fijación. En la arquitectura encontramos un perceptron multicapa (MLP) con funciones ReLU que genera deformaciones tipo PSD sobre modelos LBS, cuya aplicación en el mundo 3D garantiza la compatibilidad con los motores gráficos y su alta eficiencia. Esta metodología entrena sobre datasets públicos como AMASS Mahmood et al. (2019) y CMU MoCap, siendo capaz de manejar colisiones en poses extremas, así como colisiones cloth-to-cloth. Asimismo, permite la obtención de prendas que se ajustan a diferentes cuerpos (garment resizing). En comparación a otros métodos como TailorNet Patel et al. (2020), logra una mayor consistencia física y generalización, además de un coste computacional considerablemente menor, incluso sin GPU. Sin embargo, se ve limitado por su enfoque en poses estáticas.

2.3.2.4. SNUG

Siguiendo la tendencia hacia el aprendizaje no supervisado basado en físicas, la técnica SNUG Santesteban et al. (2022) va más allá de la deformación basada en poses. Si bien trabaja con 6,519 fotogramas de la misma base de datos, gracias a una red recurrente (cuya arquitectura no conocemos con exactitud) consigue mejores tiempos de entrenamiento y resultados más detallados que la técnica anterior. Pone el foco del aprendizaje crear una serie de funciones de loss basadas en las físicas reales de deformaciones dinámicas: “Membrane Strain Energy” (la energía transmitida entre dos puntos en base a la elasticidad del material), “Bending Energy” (el ángulo diédrico entre dos caras en relación a su propiedad de rigidez), “Gravity” (gravedad) y “Collision Penalty” (la distancia prudencial entre el modelo y el cuerpo).

2.3.2.5. Neural Cloth Simulation

Neural Cloth Simulation Bertiche et al. (2022) también propone una metodología no supervisada, unificando simulación y entrenamiento. Sigue el esquema clásico de este tipo de solvers, donde el estado de la tela depende de los instantes previos, gestionado por ventanas de frames cuyo tamaño y complejidad dependen del tipo de tela procesada. Su arquitectura consiste en un encoder-decoder recurrente que separa componentes estáticos y dinámicos, haciendo uso de modelos GRU en estos últimos y la cual permite mejorar la generalización y estabilidad del entrenamiento. Este se realiza mediante funciones de pérdida inspiradas en simulación física, incluyendo restricciones físicas en relación a deformaciones, colisiones e inercia. El dataset utilizado también es AMASS Mahmood et al. (2019), que recopila información de mas de 40.000 poses, y el modelo físico extiende la idea del modelo mass-spring Provot et al. (1995). En comparación con PBNS o SNUG, esta metodología consigue soluciones más robustas y simples que dan a simulaciones detalladas a costa de mayores tiempo de entrenamiento que sus predecesores, compensando, no obstante, la disminución de recursos al no realizar procesos de generación de datos.

Esta perspectiva va de la mano con tendencias actuales en la industria, como la ya mencionada e implementada Chaos Cloth de Unreal Engine.

2.3.2.6. HOOD: Hierarchical Graphs for Generalized Modelling of Clothing Dynamics

En investigaciones más recientes, se aborda la cuestión de la posibilidad de abstraer también la topología de la tela del aprendizaje. En este caso, HOOD utiliza GNNs, una arquitectura de envío de mensajes jerárquico y por supuesto la función de pérdida basada en la física que hemos comentado en los previos estudios [Grigorev et al. \(2023\)](#). Los graph neural networks permiten crear un gráfico que incorpora los vértices del cuerpo colisionador. A través de este se propagan los mensajes de aceleración recursivamente, lo que permite ajustar la precisión de la inferencia y adaptarse a distintos materiales, topologías y modelos de colisión.

Este estudio sigue estando limitado en aspectos como la velocidad de simulación o colisiones internas. Al fin y al cabo, este sigue siendo un campo de estudio en crecimiento, en el que todavía queda mucho por avanzar.

Capítulo 3

Descripción del Trabajo

3.1. El dataset

Para poder entrenar nuestro modelo de Inteligencia Artificial y recrear el movimiento de una tela necesitaremos un dataset que represente el comportamiento de esta.

3.1.1. La simulación

Generaremos los datasets de forma interna, creándolos a partir de simulaciones de telas grabadas por nosotros en Unity.

Nuestra simulación base se lleva a cabo en un escenario formado por un plano-tela de 25 vértices con la parte superior fija en el espacio, y una esfera que colisiona con la tela para generar el movimiento.

Para desarrollar el modelo y poder identificar errores con más facilidad, hemos reducido el tamaño de la malla a un plano simple de 6 vértices.

La bola tiene dos movimientos descritos: uno lineal y horizontal, que oscila entre dos posiciones calculadas al comenzar la simulación, y otro aleatorio, que genera caminos en tres dimensiones gracias a splines. Adicionalmente, veremos más adelante que también desactivamos el movimiento y alejamos la bola durante periodos de tiempo en la ejecución para intentar limitar la dependencia entre la tela y su distancia a la esfera.

No obstante, también hemos realizado pruebas con simulaciones más complejas como un torso con piernas simplificadas y una falda y una camiseta, con objeto de probar la efectividad de nuestro método y diferencias entre simulaciones más dinámicas y estáticas, más frecuentadas en PBD. EXTENDER O MOVER SI LO USAMOS PARA LAS PRUEBAS.

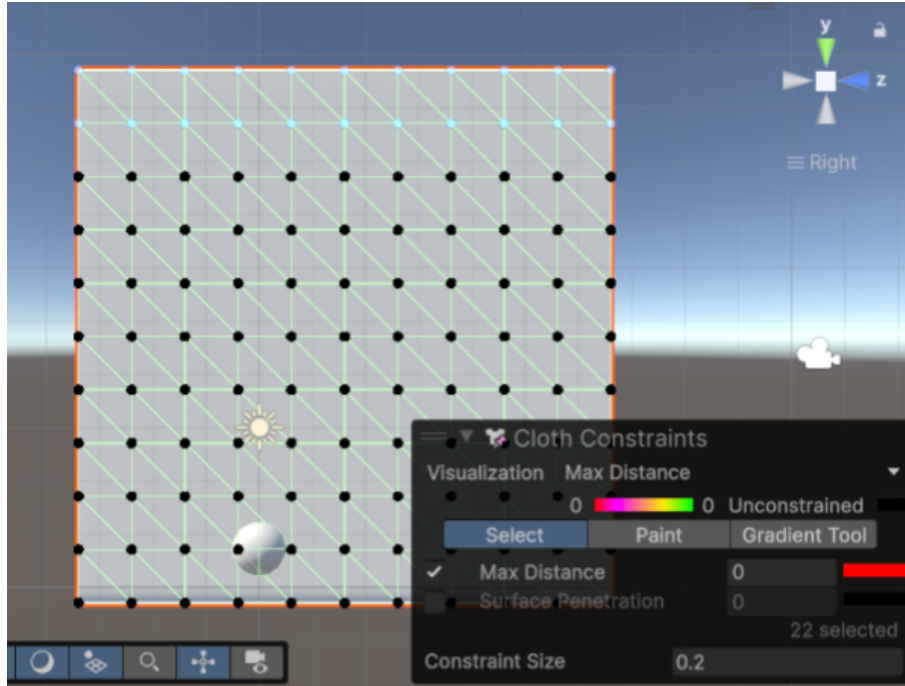


Figura 3.1: Malla de 25 vértices con parte superior fija.

3.1.2. Los parámetros

Las telas que usamos para generar las simulaciones en las que se basarán nuestros modelos son mallas que tienen un componente Cloth (y su correspondiente Skinned Mesh Renderer) y un componente de grabación ClothDatasetRecorder, encargado de guardar los datos de la simulación. Planteamos recoger diferentes parámetros que representan las características más importantes de la tela durante la simulación:

- **Posición:** las posiciones de los vértices de la tela son los parámetros básicos de las investigaciones en este campo.
- **SDF:** los *Signed Distance Field* representan la distancia entre un punto y la superficie más cercana de un objeto o conjunto de coordenadas. Son frecuentemente usados en la informática gráfica y generalmente conllevan la distancia ortogonal a un espacio métrico REVISAR DEFINICION (no sé cómo de técnica ser). En nuestro caso, representará la distancia de cada vértice al objeto colisionador, llegando a ser 0 si toca la superficie, y un valor negativo si alcanzará a traspasar. Por el momento no usamos el gradiente del SDF.
- **Velocidad:** planteamos que las velocidades de los vértices pueden ayudarnos a que el modelo sea capaz de replicar el dinamismo del movimiento.
- **Normales:** las normales de los vértices de la tela. REVISAR
- **UVs:** conservar la geometría de la tela se plantea como uno de los mayores retos para nuestra simulación. Necesitamos que el modelo comprenda cómo

están relacionados los vértices espacialmente para que no se lleven a cabo deformaciones erróneas. Ya que no comenzamos con geometrías complejas en tela o cuerpos blandos, usaremos las coordenadas de textura bidimensionales, *UVs*, para representar la superficie.

- **Máxima Distancia:** cada vértice tiene una distancia máxima a la que un vértice se puede mover desde su posición en la malla skinned. Queremos que los vértices no se pasen de su rango hábil de movimiento. Esto podemos corregirlo también en post-procesamiento.

3.1.3. Generación del dataset

A través del *ClothDatasetRecorder* realizamos “snapshots” cada intervalo de tiempo `recordTime`, que declaramos en un inicio como 3.5f. Estos datos del frame de guardan en una estructura `SnapInfo` que representa una captura del momento de tiempo de los vértices de la tela, que se añade a una lista de capturas de la simulación. Todas estas capturas se transforman en texto que se vuelca en un CSV, en el que cada línea está compuesta por la información de un frame capturado.

3.2. EntrenamientoPython

Con los datasets registrados.... EXPLICAMOS PIPELINE - Intro pytorch (environment para gpu, librerías usadas, etc) - Creación notebook - Procesamiento datos: - limpieza(quitar 1er y último frame) - normalizado de datos

3.2.1. Los modelos

Dado a que el estado del arte propone una amplia variedad de tipos de aproaches que hacer en cuando a modelos, y nosotras al trabajar a escala pequeña decidimos empezar con un modelo muy sencillo y apartir de ahí ir aumentando la complejidad, en base a los resultados problemas y limitaciones que vayamos encontrando.

El modelo inicial es un MLP...

Empezamos con un MLP simple. Problemas con los que nos hemos encontrado y qué soluciones han supuesto: Local vs Global positions Snapshots hace que pierda linea del tiempo ->max frames No tener en cuenta la linealidad del tiempo ->RNN (pasado)

Que parametros? No sabemos qué feature explota ->probar PCA Reducimos features a mínimos (posición y SDF) Random forest para ver a q parametros se les da importancia (por lo que vemos U, V y MD nada) Reducimos features a mínimos (posición y SDF) y añadimos VEL

Error acumulado: Añadimos el NOISE Una de las técnicas mas sencillas para mejorar la capacidad del modelo de corregir sus errores es entrenarlo creando esos pequeños errores artificialmente, es decir, añadiendole ruido a los datos de entrada. De esta manera, el modelo aprende a corregir estos errores y depaso los suyos propios. Las magnitudes en las que nos movemos son entre un 2 y un 5 por ciento.

Esto claramente mejoró el accuracy del modelo en las pruebas de python, pero al trasladarlo a unity seguimos con "explosiones" de la malla. Y nos dimos cuenta de que necesitábamos ir más allá (wow somos tan visionarias) eliminar el problema de base (radicales wow), cuando el error que se genera en un frame pasa a ser la base del siguiente, aunque el error estuviera en muchos casos por debajo de 0.005 metros, a una velocidad de 60FPS basta con unos segundos para que la tela deje de actuar correctamente.

ANCHORED La respuesta más intuitiva, y basada en varios papers (TODO: CITAR) consiste en coger un punto de referencia del que medir los desplazamientos. Al pensar en simulaciones de cloth, ya sea una bandera con su mastil o una prenda con su personaje, todas cuentan con unos puntos, es decir, algo a lo que anclarse. Por lo que podemos asegurarnos que la simulación se mantendrá dentro de un espacio concreto.

Guardando la posición inicial de la tela estática, calculamos las posiciones como el movimiento de la tela desde ese estado base. TODO: Eva explícalo tu. TODO: Igual deberíamos probar con la posición global??

UNROLLED Otra de las técnicas que probamos, que nos resultó mucho más exitosa fue esto [Holden et al. \(2019\)](#) TODO: Sabemos si es un RNN? Qué ostias es un RNN? pero el modelo es un RNN de 5 frames que para cada batch predice con esos 5 de input 1 frame, en 4 del input más el pred otro frame, $3in + 2pred$ otro más... y así saca hasta 5. Calcula el loss haciendo la media del real - los 5 pred y así aprende a sacar no solo un frame correcto sino un frame correcto y q sea bueno para los siguientes

Frente a un modelo base del RNN optimizamos parámetros: Windows size (pasado) Rollout steps (futuro) Noise Batch size Learning rate, tal? Epochs Mínimo dataset size para resultado visual bueno

3.3. SimulacionUnity

3.4. ModelosFuncionales

3.5. PruebaTesters

Conclusiones y Trabajo Futuro

4.1. Conclusiones

Las conclusiones de cada una de las partes del trabajo se presentan en las siguientes secciones de manera más detalladas.

4.1.1. Conclusiones de la búsqueda de datasets

Bla.

4.1.2. Conclusiones de la blablabla

Bla.

4.1.3. Limitaciones

Este estudio ha tenido una serie de limitaciones, desde blablabla Los modelos de redes neuronales no han dado los resultados esperados, esto puede ser por la falta de datos, que aunque haya un número que a simple vista parece suficiente no lo es para entrenar redes neuronales, por las limitaciones de hardware para entrenar modelos más complejos y por los tiempos requeridos para la realización de este estudio no ser lo suficientemente largos como para llevar a cabo entrenamientos más extensos y con mayor búsqueda de hiperparámetros.

Todas estas limitaciones, han llevado al siguiente plan de trabajo a futuro.

4.2. Trabajo Futuro

El trabajo a futuro de este estudio se puede centrar en varios esfuerzos:

- bla
- bla

- bla

Contribuciones Personales

Eva

Muchas cosas.

Pau

Muchas cosas.

Mik

Muchas cosas.

Bibliografía

The sciences, each straining in its own direction, have hitherto harmed us little; but some day the piecing together of dissociated knowledge will open up such terrifying vistas of reality, and of our frightful position therein, that we shall either go mad from the revelation or flee from the deadly light into the peace and safety of a new dark age.

H.P. Lovecraft, *The Call of Cthulhu*

- BERTICHE, H., MADADI, M. y ESCALERA, S. Pbns: Physically based neural simulation for unsupervised garment pose space deformation. *ACM Transactions on Graphics (TOG)*, vol. 40(6), páginas 1–14, 2021a.
- BERTICHE, H., MADADI, M. y ESCALERA, S. Neural cloth simulation. *ACM Transactions on Graphics (TOG)*, vol. 41(6), páginas 1–14, 2022.
- BERTICHE, H., MADADI, M., TYLSON, E. y ESCALERA, S. DeePSD: Automatic Deep Skinning And Pose Space Deformation For 3D Garment Animation. En *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, páginas 5451–5460. IEEE, Montreal, QC, Canada, 2021b. ISBN 978-1-6654-2812-5.
- CHAUDHARY, M., POKHARIYA, C. y NARAIN, R. Towards generalized position-based dynamics. *arXiv preprint arXiv:2511.23131*, 2025.
- CHENTANEZ, N., MACKLIN, M., MÜLLER, M., JESCHKE, S. y KIM, T. Cloth and Skin Deformation with a Triangle Mesh Based Convolutional Neural Network. *Computer Graphics Forum*, vol. 39(8), páginas 123–134, 2020. ISSN 0167-7055, 1467-8659.
- CONTRIBUTORS, P. Pytorch documentation. <https://docs.pytorch.org/docs/stable/index.html>, 2026. Accessed: April 17, 2026.
- GAMES, E. Chaos cloth. <https://dev.epicgames.com/documentation/unreal-engine/machine-learning-cloth-simulation-overview>, 2025. Accessed: (17/04/2026).

- GRIGOREV, A., THOMASZEWSKI, B., BLACK, M. J. y HILLIGES, O. HOOD: Hierarchical Graphs for Generalized Modelling of Clothing Dynamics. 2023. ArXiv:2212.07242 [cs].
- HAVOK. Havok physics. <https://www.havok.com/havok-physics/>, 2026. Accessed: 2026-04-28.
- HECKER, C. Physics in computer games (title only). *Communications of the ACM*, vol. 43(7), páginas 34–39, 2000.
- HOLDEN, D., DUONG, B. C., DATTA, S. y NOWROUZEZAHRAI, D. Subspace neural physics: fast data-driven interactive simulation. En *Proceedings of the 18th annual ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, páginas 1–12. ACM, Los Angeles California, 2019. ISBN 978-1-4503-6677-9.
- JIM HUGUNIN, U. Unite 2016 - gpu accelerated high resolution cloth simulation. <https://www.youtube.com/watch?v=kCGHX1LR318>, 2016. Accessed: (15/04/2026).
- KENNY ERLEBEN, K. H., JON SPORRING. *Physics-based Animation*. Charles River Media, 2005.
- MACKLIN, M., MÜLLER, M. y CHENTANEZ, N. XPBD: position-based simulation of compliant constrained dynamics. En *Proceedings of the 9th International Conference on Motion in Games*, páginas 49–54. ACM, Burlingame California, 2016. ISBN 978-1-4503-4592-7.
- MAHMOOD, N., GHORBANI, N., TROJE, N. F., PONS-MOLL, G. y BLACK, M. J. Amass: Archive of motion capture as surface shapes. páginas 5442–5451, 2019.
- METHOD, V. Obi cloth. <https://assetstore.unity.com/packages/tools/physics/obi-cloth-81333?aid=1011134eQ>, 2019. Accessed: (16/04/2026).
- MILLINGTON, I. *Game Physics Engine Development*. CRC Press, 2007. ISBN 9781482267327.
- MÜLLER, M., HEIDELBERGER, B., HENNIX, M. y RATCLIFF, J. Position based dynamics. *Journal of Visual Communication and Image Representation*, vol. 18(2), páginas 109–118, 2007. ISSN 10473203.
- NVIDIA. Physx sdk. <https://developer.nvidia.com/physx-sdk>, 2026. Accessed: 2026-04-9.
- PATEL, C., LIAO, Z. y PONS-MOLL, G. Tailornet: Predicting clothing in 3d as a function of human pose, shape and garment style. En *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, páginas 7365–7375. 2020.
- PROJECT, O. Onnx: Open neural network exchange. <https://onnx.ai/>, 2026. Accessed: 2026-04-17.

- PROVOT, X. Deformation Constraints in a Mass Spring Model to Describe Rigid Cloth Behavior. 1995.
- PROVOT, X. ET AL. Deformation constraints in a mass-spring model to describe rigid cloth behaviour. En *Graphics interface*, páginas 147–147. Canadian Information Processing Society, 1995.
- SALEN, K. y ZIMMERMAN, E. *Rules of Play: Game Design Fundamentals*. MIT Press, 2004.
- SANTESTEBAN, I., OTADUY, M. A. y CASAS, D. Snug: Self-supervised neural dynamic garments. En *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, páginas 8140–8150. 2022.
- TECHNOLOGIES, U. Physics integrations. <https://docs.unity3d.com/Manual/physics-integrations.html>, 2026a. Accessed: 2026-04-28.
- TECHNOLOGIES, U. Unity sentis: Ai inference package v2.4. <https://docs.unity3d.com/Packages/com.unity.ai.inference@2.4/manual/index.html>, 2026b. Accessed: 2026-04-17.
- TOWER, S. The big game engines report. https://app.sensortower.com/vgi/assets/reports/The_Big_Game_Engines_Report_of_2025.pdf, 2025. Accessed: 2026-04-28.
- VA, H., CHOI, M.-H. y HONG, M. Real-time cloth simulation using compute shader in unity3d for ar/vr contents. *Applied Sciences*, vol. 11(17), 2021. ISSN 2076-3417.

Carteles para la generación del dataset



Figura A.1: Cartel colgado en la facultad de informática

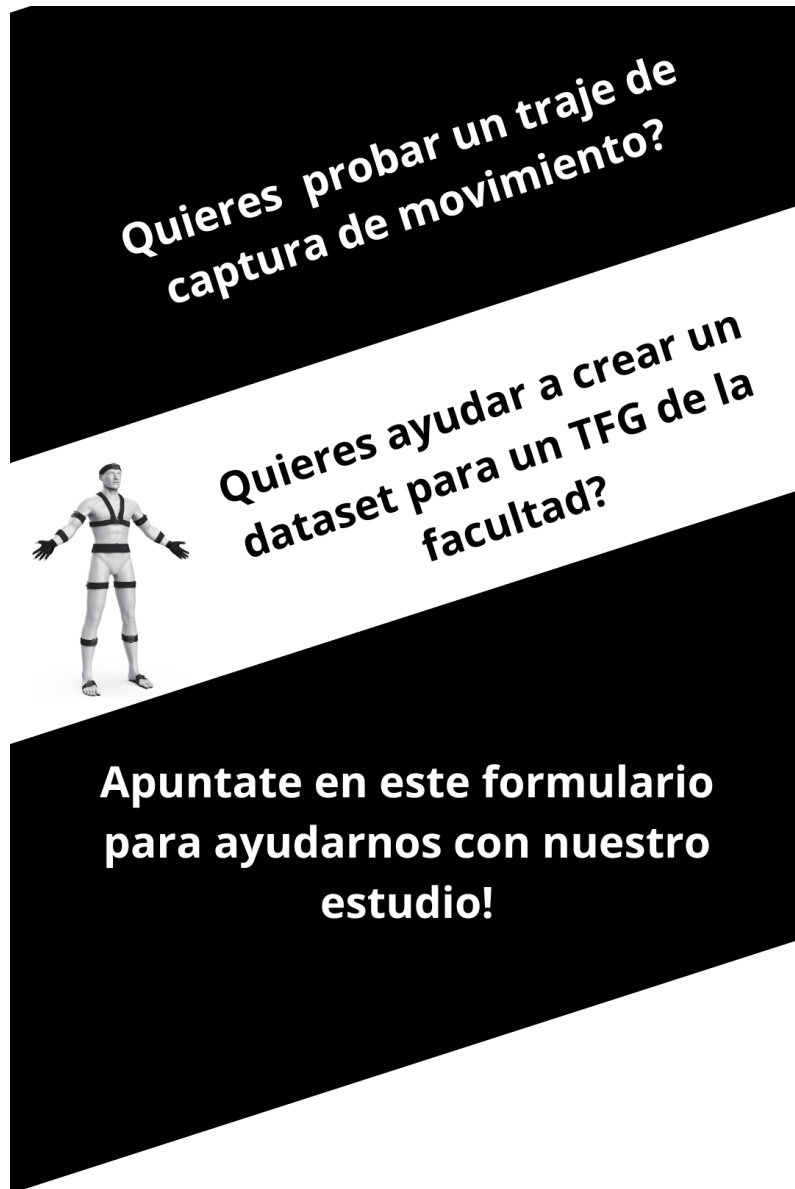


Figura A.2: Cartel colgado en redes sociales



Figura A.3: Cartel colgado en pantallas de la facultad

Formulario de recogida de citas

Participación Dataset para detección de poses

Necesitamos datos para un TFG de la Facultad de Informática de la UCM. Lo importante, **¿qué datos recopilamos?** La siguiente lista de datos son los datos que vamos a recopilar y hacer públicos en el propio estudio del tfg o en un dataset público en [kaggle](#):

- Edad
- Sexo
- Nacionalidad
- Datos de coordenadas de distintas partes del cuerpo dadas por un traje de captura de movimiento

Estos datos se recogerán en un formulario a parte el día de la prueba, siendo hechos desde un correo de los organizadores y permaneciendo totalmente anónimos sin correlación a la persona que ha realizado la prueba.

OBJETIVO DE LOS DATOS: los datos de edad, nacionalidad y sexo son meramente estadísticos, solo se usarán para demostrar la heterogeneidad (o falta de) de los datos tomados. Los datos de coordenadas se usarán para entrenar distintos modelos de clasificación de pose con la intención de poder decir con acierto el gesto o pose que está haciendo una persona mientras lleva el traje puesto.

LOCALIZACIÓN DE LA PRUEBA: [Facultad de Informática \(UCM\)](#), Calle del Prof. José García Santesmases, 9, Moncloa - Aravaca, 28040 Madrid

alejba02@ucm.es [Cambiar de cuenta](#)

* Indica que la pregunta es obligatoria

Correo *

Tu dirección de correo electrónico 

¿Aceptas la recogida de datos y la participación en el muestreo? *

☐ Sí

☐ No

Siguiente

Página 1 de 2

Borrar formulario

Este formulario se creó en Universidad Complutense de Madrid.
¿Parece sospechoso este formulario? [Informe](#)



Figura B.1: Formulario de recogida de citas (primera parte)

Participación Dataset para detección de poses

alejba02@ucm.es [Cambiar de cuenta](#)

Horario disponible

A continuación marca las horas en las podrías estar disponible. Con la respuesta que proporciones te mandaremos un correo con un día y hora determinados.

Lunes

- ☐ 9:00-10:00
- ☐ 10:00-11:00
- ☐ 11:00-12:00
- ☐ 12:00-13:00
- ☐ 13:00-14:00
- ☐ 14:00-15:00
- ☐ 15:00-16:00
- ☐ 16:00-17:00
- ☐ 17:00-18:00
- ☐ 18:00-19:00
- ☐ 19:00-20:00

Figura B.2: Formulario de recogida de citas (segunda parte)

Resultados de los distintos modelos CNN

La línea verde en los gráficos de tensorboard indican la mejor combinación de hiperparámetros encontrada en cada caso

C.1. Intervalo 0.2s

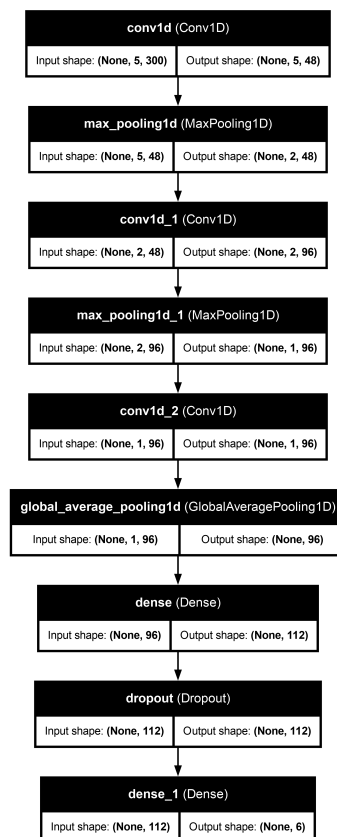


Figura C.1: Esquema del modelo CNN con 0.2s de intervalo

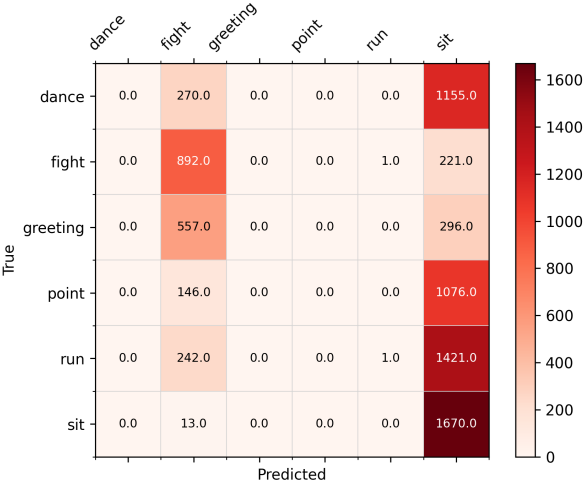


Figura C.2: Matriz de confusión del modelo CNN con 0.2s de intervalo

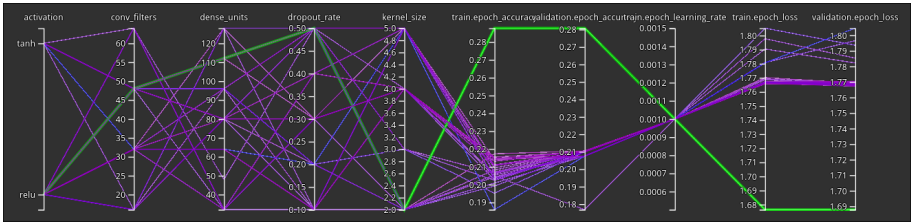


Figura C.3: Gráfico de entrenamiento del modelo CNN con 0.2s de intervalo (mejor val_accuracy = 0.28777)

C.2. Intervalo 0.4s

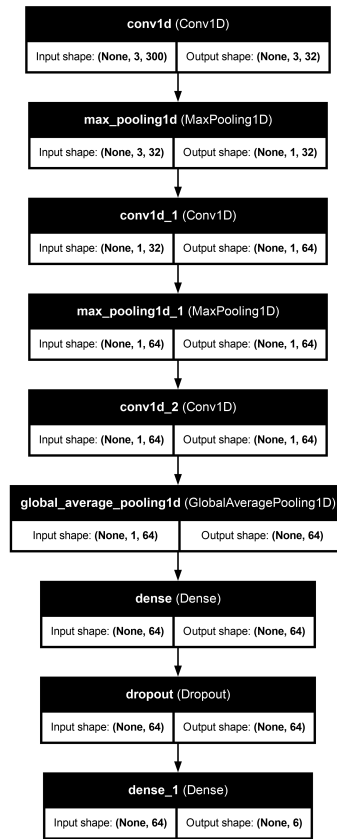


Figura C.4: Esquema del modelo CNN con 0.4s de intervalo

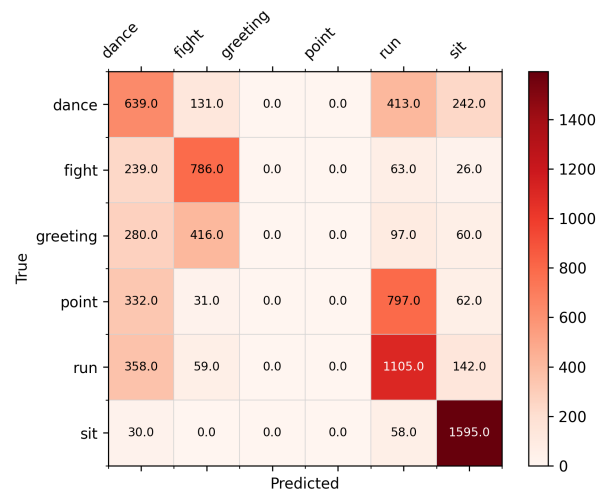


Figura C.5: Matriz de confusión del modelo CNN con 0.4s de intervalo

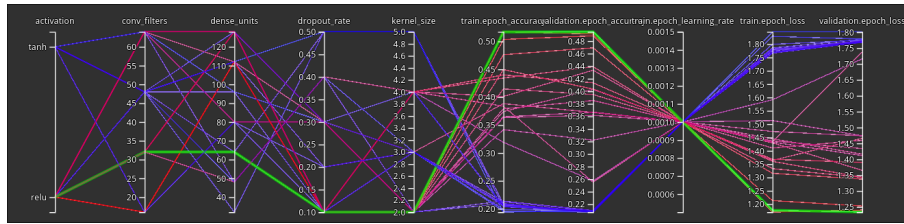


Figura C.6: Gráfico de entrenamiento del modelo CNN con 0.4s de intervalo (mejor $\text{val_accuracy} = 0.49473$)

C.3. Intervalo 0.6s

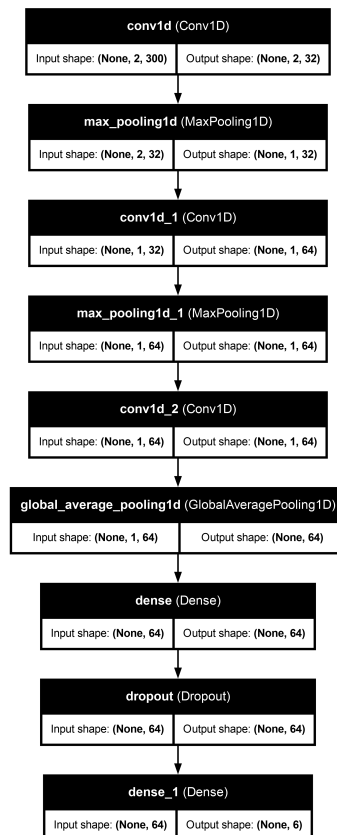


Figura C.7: Esquema del modelo CNN con 0.6s de intervalo

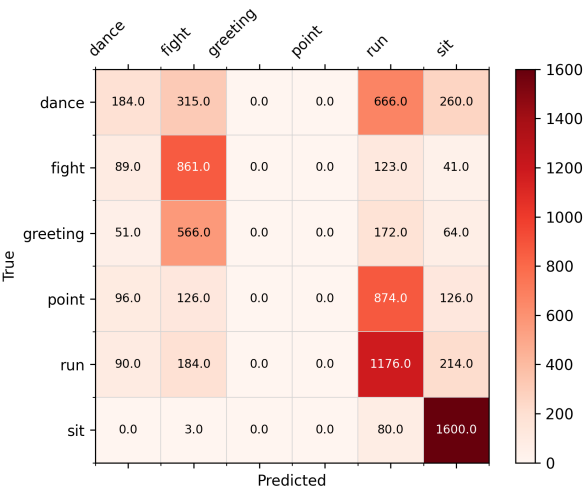


Figura C.8: Matriz de confusión del modelo CNN con 0.6s de intervalo

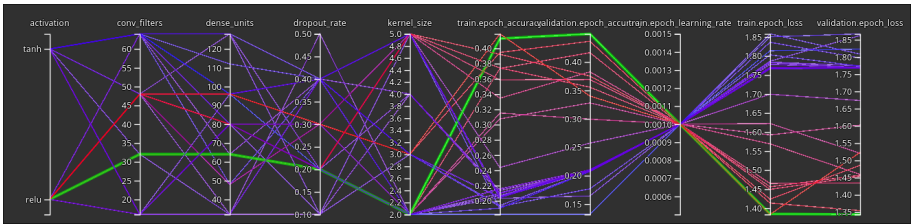


Figura C.9: Gráfico de entrenamiento del modelo CNN con 0.6s de intervalo (mejor val_accuracy = 0.44952)

C.4. Intervalo 0.8s

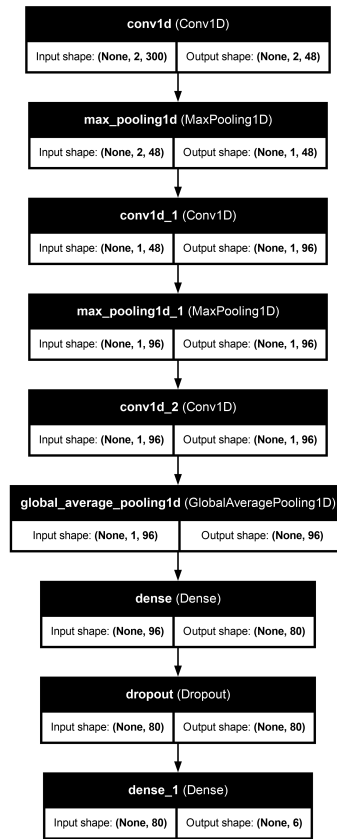


Figura C.10: Esquema del modelo CNN con 0.8s de intervalo

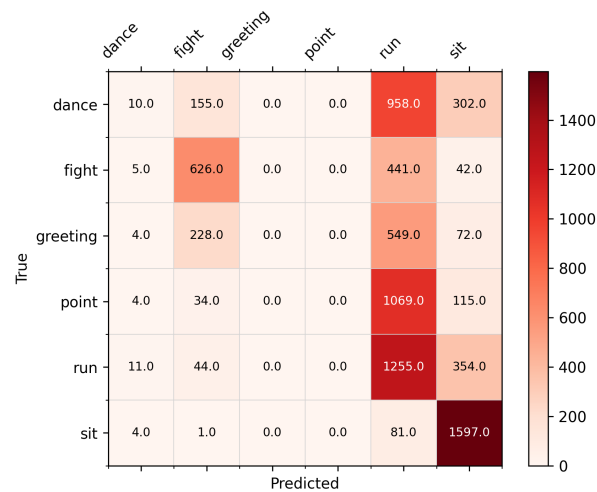


Figura C.11: Matriz de confusión del modelo CNN con 0.8s de intervalo

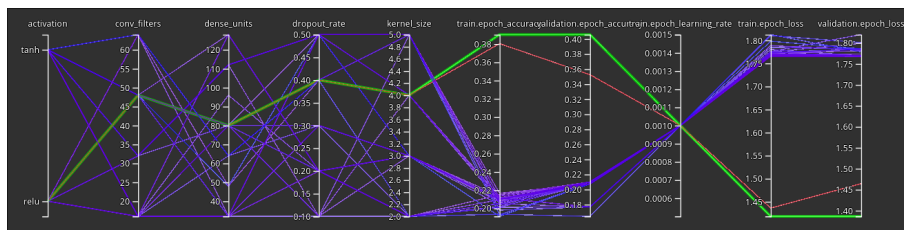


Figura C.12: Gráfico de entrenamiento del modelo CNN con 0.8s de intervalo (mejor val_accuracy = 0.40583)

C.5. Intervalo 1.0s

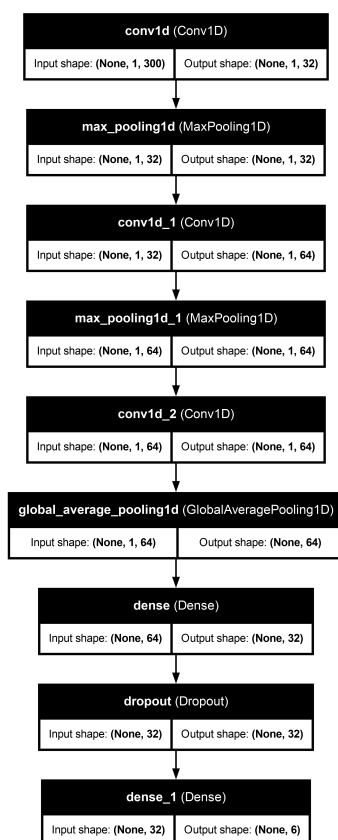


Figura C.13: Esquema del modelo CNN con 1.0s de intervalo

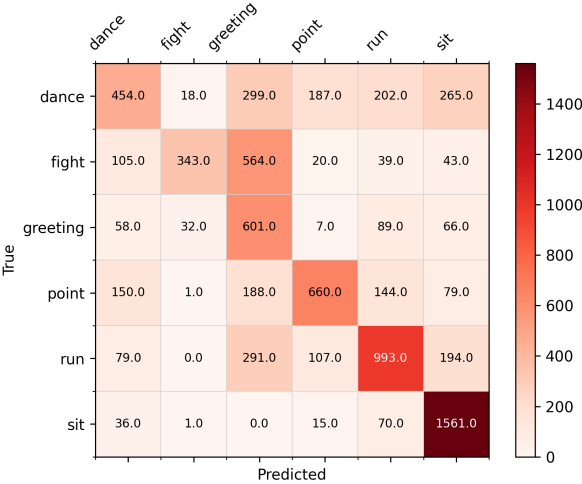


Figura C.14: Matriz de confusión del modelo CNN con 1.0s de intervalo

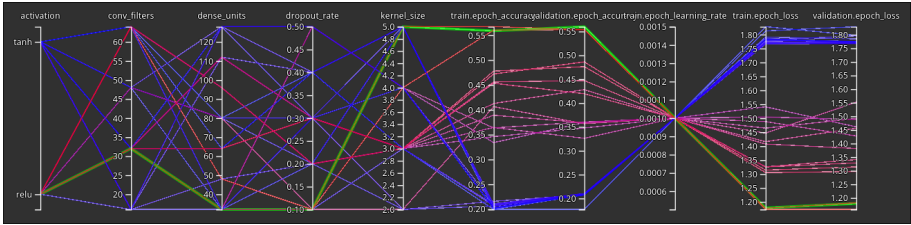


Figura C.15: Gráfico de entrenamiento del modelo CNN con 1.0s de intervalo (mejor val_accuracy = 0.56002)

Glosario

CNN Convolutional Neural Network

Licencia de uso del documento

Esta obra está bajo una licencia [Creative Commons](http://creativecommons.org/licenses/by-sa/4.0/legalcode) «Atribución-NoComercial-CompartirIgual 4.0 Internacional».



<http://creativecommons.org/licenses/by-sa/4.0/legalcode>.

Licencia de uso del código fuente

Los archivos de código fuente para generar este documento se encuentran en <https://github.com/FratosVR/Memoria-TFG> bajo la licencia [GPL-3.0](#)